

P2363R0

Extending associative containers with the remaining heterogeneous overloads

Published Proposal, 2021-04-15

This version:

<http://wg21.link/P2363R0>

Authors:

[Konstantin Boyarinov](#) (Intel)

[Sergey Vinogradov](#) (Intel)

[Ruslan Arutyunyan](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

The authors propose heterogeneous overloads for the remaining member functions in ordered and unordered associative containers.

Table of Contents

1 Motivation

2 Prior work

3 Proposal overview

3.1 `try_emplace`

3.2 `insert_or_assign`

3.3 `operator[]`

3.4 `bucket`

3.5 `at`

3.6 `insert`

3.7 Proposed API summary

4 Plans

References

Informative References

§ 1. Motivation

[N3657] and [P0919R0] introduced heterogeneous lookup support for ordered and unordered associative containers in C++ Standard Library. [P2077R2] proposed heterogeneous erasure overloads. As a result, a temporary key object is not created when a different (but comparable) type is provided as a key to the member function. But there are still no heterogeneous overloads for methods such as `insert_or_assign`, `operator[]` and others.

Consider the following example:

```
void foo(std::map<std::string, int, std::less<void>>& m) {  
    const char* lookup_str = "Lookup_str";  
    auto it = m.find(lookup_str); // matches to template overload  
    // ...  
    auto result = m.insert_or_assign(lookup_str, 1); // no heterogeneous alternative  
}
```

Function `foo` accepts a reference to the `std::map<std::string, int>` object. A comparator for the map is `std::less<void>`, which provides `is_transparent` valid qualified-id, so the `std::map` allows using heterogeneous overloads with `Key` template parameter for lookup functions, such as `find`, `upper_bound`, `equal_range`, etc.

In the example above, the `m.find(lookup_str)` call does not create a temporary object of the `std::string` to perform a lookup. But the `m.insert_or_assign(lookup_str, 1)` call causes implicit conversion from `const char*` to `std::string` even if the insertion will not take place. The allocation and construction (as well as subsequent destruction and deallocation) of the temporary object of `std::string` or any custom object might be quite expensive and reduce the program performance.

All the proposed APIs in this paper allow to avoid mentioned performance penalty.

§ 2. Prior work

Heterogeneous lookup overloads for ordered and unordered associative containers were added into C++ standard. For example:

```
template <typename K>  
iterator find(const K& x);
```

The corresponding overloads were added for `count`, `contains`, `equal_range`, `lower_bound` and `upper_bound` member functions.

[P2077R2] proposed heterogeneous erasure overloads for ordered and unordered associative containers:

```
template <typename K>  
size_type erase(K&& x);  
  
template <typename K>  
node_type extract(K&& x);
```

Constraints:

- qualified-id `Compare::is_transparent` is valid and denotes a type for ordered associative containers
- qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types for unordered associative containers

where `Compare` is a type of comparator passed to an ordered container, `Hash` and `Pred` are types of hash function and key equality predicate passed to an unordered container respectively.

§ 3. Proposal overview

We propose to add heterogeneous overloads for the following member functions:

- `insert` in `std::set` and `std::unordered_set`
- `insert_or_assign`, `try_emplace`, `operator[]`, `at` in `std::map` and `std::unordered_map`
- `bucket` in `std::unordered_set`, `std::unordered_map`, `std::unordered_multiset` and `std::unordered_multimap`

§ 3.1. try_emplace

We propose the following API (`std::map` and `std::unordered_map`):

```
template <typename K, typename... Args>
std::pair<iterator, bool> try_emplace(K&& k, Args&&... args);

template <typename K, typename... Args>
iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
```

Effects: If the map already contains an element whose key compares equivalent with `k`, there is no effect. Otherwise, inserts an object of type `value_type` constructed with `std::piecewise_construct`, `std::forward_as_tuple(std::forward<K>(k))`, `std::forward_as_tuple(std::forward<Args>(args)...)...`.

Constraints:

1. For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types
2. `std::is_constructible_v<value_type, std::piecewise_construct_t, std::tuple<K&&>, std::tuple<Args&&...>>` is true.
3. `std::is_convertible_v<K&&, const_iterator>` and `std::is_convertible_v<K&&, iterator>` are both false.

Constraints 2 and 3 are introduced to maintain backward compatibility.

Constraint 2 makes homogeneous overload to be called when `K&&` is convertible to `key_type` but `value_type` is not constructible from `std::piecewise_construct`, `std::forward_as_tuple(std::forward<K>(k))`, `std::forward_as_tuple(std::forward<Args>(args)...)...`.

Constraint 3 makes homogeneous overload to be called when `K&&` is convertible to `const_iterator` or to `iterator`.

§ 3.2. insert_or_assign

We propose the following API (`std::map` and `std::unordered_map`):

```
template <typename K, typename M>
std::pair<iterator, bool> insert_or_assign(K&& k, M&& obj);

template <typename K, typename M>
iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);
```

Effects: If the map already contains an element `e` whose key compares equivalent with `k`, assigns `std::forward<M>(obj)` to `e.second`. Otherwise, inserts an object of type `value_type` constructed with `std::forward<K>(k)`, `std::forward<M>(obj)`.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types
- `std::is_constructible_v<value_type, K&&, M&&>` is true.

Backward compatibility problem in case of `K&&` convertibility to `iterator` or `const_iterator` will not take place because the number of arguments is fixed - call with three arguments would always consider the overloads with `hint` only.

§ 3.3. operator[]

We propose the following API (`std::map` and `std::unordered_map`):

```
template <typename K>
mapped_type& operator[](K&& k);
```

Effects: Equivalent to: `return try_emplace(std::forward<K>(k)).first->second.`

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

We do not introduce the following constraint: `std::is_constructible_v<value_type, std::piecewise_construct_t, K&&> == true` because `K&&` is used as an argument for `value_type` constructor anyway. We do not consider the use-case when the `K&&` is convertible to `key_type`, but `key_type` is not constructible from `K&&` as important to maintain.

§ 3.4. bucket

We propose the following API (`std::unordered_map`, `std::unordered_multimap`, `std::unordered_set` and `std::unordered_multiset`):

```
template <typename K>
size_type bucket(const K& k) const;
```

Returns: The index of the bucket in which elements with keys compared equivalent with `k` would be found, if any such element existed.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

§ 3.5. at

We propose the following API (`std::map` and `std::unordered_map`):

```
template <typename K>
mapped_type& at(const K& k);

template <typename K>
const mapped_type& at(const K& k) const;
```

Returns: A reference to the `mapped_type` corresponding to `k` in `*this`.

Throws: An exception object of type `std::out_of_range` if no such element is present.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

§ 3.6. insert

We considered to add heterogeneous overloads of `insert` for all associative containers, but found the applicability only for `std::set` and `std::unordered_set` with the following API:

```
template <typename K>
std::pair<iterator, bool> insert(K&& k);

template <typename K>
iterator insert(const_iterator hint, K&& k);
```

Effects: If the set already contains an element which compares equivalent with `k`, there is no effect. Otherwise, inserts an object constructed with `std::forward<K>(k)` into the set.

Constraints:

- For ordered associative containers: qualified-id `Compare::is_transparent` is valid and denotes a type For unordered associative containers: qualified-ids `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types

We do not introduce the following constraint: `std::is_constructible_v<value_type, K&&> == true`. because `K&&` is used as an argument for `value_type` constructor anyway. We do not consider the use-case when the `K&&` is convertible to `key_type`, but `key_type` is not constructible from `K&&` as important to maintain.

Adding heterogeneous `insert` overload makes no sense for associative containers with non-unique keys (`std::multimap`, `std::multiset`, `std::unordered_multimap` and `std::unordered_multiset`) because the insertion will be successful in any case and the key would be always constructed. All additional overheads introduced by `insert` can be mitigated by using `emplace`.

For `std::map` and `std::unordered_map`, heterogeneous insertion also makes no sense because heterogeneous `try_emplace` covers the relevant use cases.

§ 3.7. Proposed API summary

The proposed heterogeneous overloads for various methods are summarized in the table below:

Member function	std::set	std::multiset	std::map	std::multimap	std::unordered_set	std::unordered_multiset	std::unor
insert	K&&	Not applicable	Not applicable	Not applicable	K&&	Not applicable	Not applic
insert_or_assign	Not available	Not available	K&&	Not available	Not available	Not available	K&&
try_emplace	Not available	Not available	K&&	Not available	Not available	Not available	K&&
operator[]	Not available	Not available	K&&	Not available	Not available	Not available	K&&
bucket	Not available	Not available	Not available	Not available	const K&	const K&	const K&
at	Not available	Not available	const K&	Not available	Not available	Not available	const K&

§ 4. Plans

- Provide implementation experience results
- Provide performance measurements
- Prepare the wording

§ References

§ Informative References

[N3657]

J. Wakely, S. Lavavej, J. Muñoz. [Adding heterogeneous comparison lookup to associative containers \(rev 4\)](https://wg21.link/n3657). 19 March 2013. URL: <https://wg21.link/n3657>

[P0919R0]

Mateusz Pusz. [Heterogeneous lookup for unordered containers](https://wg21.link/p0919r0). 8 February 2018. URL: <https://wg21.link/p0919r0>

[P2077R2]

Konstantin Boyarinov, Sergey Vinogradov; Ruslan Arutyunyan. [Heterogeneous erasure overloads for associative containers](https://wg21.link/p2077r2). 15 December 2020. URL: <https://wg21.link/p2077r2>